

Programação Orientada a Objetos — Respostas

1. Fundamentos da Orientação a Objetos

- A **Programação Orientada a Objetos (POO)** é um paradigma de programação que organiza o software em torno de **objetos**. Os principais conceitos da POO são **abstração, encapsulamento, herança e polimorfismo**.

Um objeto reúne **estado** (dados/atributos) e **comportamentos** (métodos), permitindo que dados e operações relacionadas permaneçam próximos.

- Na programação **procedural**, o foco principal está em procedimentos e funções que manipulam dados. Ou seja, uma separação em estruturas de dados e funções.
- **POO, procura-se organizar o sistema em objetos que sejam responsáveis pelos próprios dados e comportamentos.**

Por exemplo, em um sistema bancário procedural, poderíamos ter uma estrutura `conta` e funções como `depositar(conta, valor)` e `sacar(conta, valor)`. Em POO, poderíamos ter uma classe `ContaBancaria` contendo o saldo e métodos `depositar()` e `sacar()`.

A POO pode contribuir para sistemas grandes porque:

- **Organiza** o código em unidades.
- **Facilita a manutenção**, alterações podem ficar restritas a determinadas classes.
- **Favorece a reutilização**, por meio de composição, herança e componentes abstraídos.
- **Permite evolução**, utilizando interfaces, abstrações e polimorfismo para substituir ou adicionar implementações.
- **Reduz o acoplamento**, quando as classes dependem de abstrações em vez de detalhes concretos.
- **Aumenta a coesão**, quando cada classe concentra responsabilidades relacionadas.

OBS: Entretanto, POO não garante automaticamente uma boa arquitetura. Uma aplicação pode utilizar muitas classes, heranças e padrões e ainda ser difícil de compreender. A qualidade depende de como os conceitos são aplicados.

2. Classes e objetos

Uma **classe** é uma abstração que descreve características e comportamentos que determinados objetos devem possuir. Um **objeto** é uma instância concreta dessa classe

Classe = o modelo/molde.

Objeto = algo criado a partir desse molde.

3. Abstração

Abstração consiste em representar os aspectos essenciais de uma entidade para determinado contexto, ignorando detalhes que não são necessários.

> Um objeto do mundo real possui inúmeras características, mas um sistema geralmente precisa trabalhar apenas com algumas delas.

Considere uma entidade "Pessoa".

Exemplos de Abstração

- Em um sistema universitário, uma pessoa poderia ser representada por:

nome
CPF
matrícula
curso
disciplinas

- Já em um sistema hospitalar, a representação poderia incluir:

nome
CPF
data de nascimento
tipo sanguíneo
alergias
histórico médico

A mesma pessoa possui características reais muito mais numerosas, mas cada sistema seleciona as informações necessárias para cumprir seus objetivos.

Assim, abstração permite reduzir a complexidade. O programador trabalha com um modelo simplificado da realidade, adequado ao problema que precisa resolver.

4. Encapsulamento

Encapsulamento consiste em organizar estado e comportamento dentro de uma unidade, normalmente uma classe, controlando como o estado interno pode ser acessado ou

O encapsulamento está relacionado ao **controle de acesso**, às invariantes do objeto e à manutenção. Quando os detalhes internos ficam protegidos, a implementação pode ser modificada sem exigir mudanças em todos os consumidores da classe.

5. Encapsulamento e regras de negócio

- Permitir alterações diretas no atributo `saldo` é perigoso porque qualquer componente poderia realizar operações inválidas:

```
conta.saldo = -5000
```

ou modificar o saldo sem registrar uma operação.

Uma abordagem orientada a objetos é manter o saldo sob responsabilidade da própria conta:

```
class ContaBancaria:
    def __init__(self):
        self._saldo = 0

    def depositar(self, valor):
        if valor <= 0:
            raise ValueError("Depósito inválido")
        self._saldo += valor

    def sacar(self, valor):
        if valor <= 0:
            raise ValueError("Saque inválido")
        if valor > self._saldo:
            raise ValueError("Saldo insuficiente")
        self._saldo -= valor

    def consultar_saldo(self):
        return self._saldo
```

Uma transferência poderia ser implementada como uma operação que coordena duas contas:

```
def transferir(origem, destino, valor):
```

```
origem.sacar(valor)
destino.depositar(valor)
```

Em sistemas reais, regras adicionais poderiam envolver limites, autenticação, transações, tarifas e registro de operações.

O ponto fundamental é que **as regras relacionadas ao estado da conta devem ser protegidas contra alterações arbitrárias.**

6. Herança

- **Herança** permite criar uma classe baseada em outra, reutilizando as características e comportamentos da classe principal.

A classe original é chamada de superclasse, classe-base ou classe-pai.

A classe derivada é chamada de subclasse.

Exemplo:

```
class Funcionario: (Classe principal)
    def trabalhar(self):
        print("Trabalhando")

class Professor(Funcionario): (Sub-classe)
    def lecionar(self):
        print("Lecionando")
```

Professor herda de **Funcionário**. (Herança)

As vantagens incluem:

- reutilização de código;
- representação de relações de especialização;
- possibilidade de polimorfismo;
- compartilhamento de comportamentos comuns.

> Porém, herança cria uma relação relativamente forte entre as classes. Alterações na superclasse podem afetar subclasses.

> Ela pode ser inadequada quando a relação não representa realmente uma relação verdadeira

Por exemplo, **Carro** não deveria necessariamente herdar de **Motor** apenas porque um carro possui um motor. Nesse caso, **composição** é mais apropriada.

7. Polimorfismo

Polimorfismo significa, de maneira geral, permitir que diferentes objetos sejam utilizados por meio de uma mesma abstração ou operação, apresentando comportamentos específicos. **A mesma chamada produz comportamentos diferentes.**

O **polimorfismo por substituição** permite que uma subclasse seja utilizada onde uma superclasse ou abstração é esperada.

A **sobreposição (overriding)** ocorre quando uma subclasse fornece sua própria implementação de um método herdado.

Existem ainda outras formas de polimorfismo, como parametrização por tipos genéricos e mecanismos de sobrecarga, dependendo da linguagem.

8. Polimorfismo e extensibilidade

Podemos definir uma abstração comum:

```
class Pagamento:
    def processar(self, valor):
        raise NotImplementedError
```

Implementações:

```
class Cartao(Pagamento):
    def processar(self, valor):
        print("Processando cartão")
```

```
class Pix(Pagamento):
    def processar(self, valor):
        print("Processando PIX")
```

```
class Boleto(Pagamento):
    def processar(self, valor):
        print("Processando boleto")
```

```
class Transferencia(Pagamento):
    def processar(self, valor):
        print("Processando transferência")
```

O processamento principal pode ser:

```
def realizar_pagamento(pagamento, valor):
    pagamento.processar(valor)
```

Esse código não precisa conhecer os detalhes de cada forma de pagamento.

Para adicionar uma nova modalidade, como **Criptomoeda**, pode-se criar uma nova implementação da abstração sem alterar necessariamente o código de processamento.

Isso reduz dependências e favorece a extensibilidade.

9. Sobrecarga e sobrescrita

- **Sobrecarga (overloading)** ocorre quando métodos possuem o mesmo nome, mas diferentes assinaturas, permitindo operações semelhantes com diferentes parâmetros. **A sobrecarga pode oferecer diferentes formas de chamar uma operação**

Por exemplo:

```
calcular(int valor)
calcular(int valor, int desconto)
```

Exemplo:

```
def calcular(valor, desconto=0):
    return valor - desconto
```

- **Sobrescrita (overriding)** ocorre quando uma subclasse redefine um método herdado. **A sobrescrita permite especializar um comportamento herdado.**

```
class Animal:
    def falar(self):
        print("Som")
```

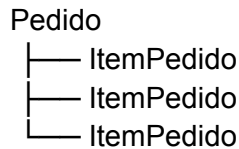
```
class Cachorro(Animal):
    def falar(self):
        print("Au au")
```

OBS: O uso excessivo de qualquer mecanismo pode prejudicar legibilidade.

10. Composição e agregação

- **Composição** representa o objeto composto que normalmente controla o ciclo de vida de seus componentes.

Por exemplo:



Os itens fazem parte do pedido.

- **Agregação** representa uma relação que os objetos participantes podem existir independentemente.

Exemplo:

Departamento — Professor

Um professor pode existir mesmo que deixe de pertencer a determinado departamento.

OBS: A composição frequentemente é preferível porque permite montar comportamentos a partir de componentes sem criar hierarquias rígidas.

11. Classes em Python

Uma classe é declarada utilizando `class`:

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco

    def aplicar_desconto(self, percentual):
        self.preco *= (1 - percentual / 100)
```

Um objeto é criado chamando a classe:

```
produto = Produto("Notebook", 3000)
```

`__init__()` = método que prepara/inicializa o objeto quando ele é criado.

O parâmetro `self` representa um ponteiro onde vai puxar a classe principal.

Nesse exemplo:

- `Produto` é a classe.
 - `nome` e `preco` são atributos
 - `__init__()` inicializa o objeto.
 - `aplicar_desconto()` é um comportamento.
 - `produto` é uma instância de `Produto`.
-

12. Encapsulamento em Python

No Python, o controle de acesso aos atributos é mais baseado em **convenções** do que em bloqueios rígidos.

Atributo público: pode ser acessado e alterado normalmente por qualquer parte do programa.

Atributo com convenção de protegido: é identificado por um sublinhado no início do nome. Isso indica que o atributo deve ser tratado como **interno da classe** e que seu acesso externo deve ser evitado. É uma convenção, não uma proibição.

Atributo com name mangling: é identificado por dois sublinhados no início. O Python modifica internamente o nome do atributo para dificultar seu acesso direto e evitar conflitos de nomes, principalmente em herança.

Portanto, o encapsulamento em Python é baseado principalmente em **convenções, interfaces, propriedades e design da classe**, e não em mecanismos rígidos de segurança.

13. Herança em Python

Podemos estruturar a hierarquia:

```
class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario

    def apresentar(self):
        print(self.nome)
```

```
class Professor(Funcionario):
```

```
def lecionar(self):
    print("Ministrando aula")

class Administrador(Funcionario):
    def administrar(self):
        print("Administrando")
```

As subclasses reutilizam os atributos e métodos de **Funcionario**.

Quando precisam especializar a inicialização, podem utilizar **super()**:

```
class Professor(Funcionario):
    def __init__(self, nome, salario, disciplina):
        super().__init__(nome, salario)
        self.disciplina = disciplina
```

Também podem sobrescrever métodos:

```
class Professor(Funcionario):
    def apresentar(self):
        print(f"Professor: {self.nome}")
```

OBS: Assim, a classe-pai concentra elementos comuns, enquanto as subclasses adicionam ou especializam comportamentos.

14. Polimorfismo em Python

Python utiliza **tipagem dinâmica** e permite polimorfismo sem exigir necessariamente uma hierarquia formal de classes.

Um conceito importante é o **duck typing**. Se um objeto possui os comportamentos necessários, ele pode ser utilizado.

O duck typing desloca o foco da identidade do tipo para o **comportamento**

OBS: Isso proporciona flexibilidade, embora também exija cuidado com contratos e testes, já que alguns erros podem aparecer em tempo de execução.

15. Métodos especiais (dunder methods)

Métodos especiais possuem nomes cercados por dois sublinhados e permitem integrar objetos definidos pelo programador ao comportamento da linguagem.

Esses métodos permitem que objetos personalizados participem da sintaxe e das operações nativas da linguagem de maneira consistente.

`__init__()` = Inicializa uma instância

`__str__()` = Define uma representação

`__repr__()` = Fornece uma representação destinada principalmente a depuração e desenvolvimento.

`__eq__()` = Define como objetos são comparados com `==`.

`__len__()` = Permite que `len(objeto)` funcione

16. Classes abstratas em Python

Uma **classe abstrata** define uma abstração que não deve necessariamente ser instanciada diretamente e estabelece operações que suas subclasses devem implementar.

O módulo `abc` do Python permite criar **classes abstratas**, que servem como modelos ou contratos para outras classes.

Uma **classe abstrata** define quais comportamentos suas classes filhas devem possuir.

Um **método abstrato** determina uma operação que as subclasses precisam implementar.

OBS: o `abc` ajuda a garantir que diferentes classes sigam as mesmas regras e ofereçam os comportamentos necessários, tornando o sistema mais organizado, previsível e fácil de manter.

17. Múltipla herança em Python

Uma vantagem de **Herança múltipla** é combinar comportamentos de diferentes classes.

Porém, múltipla herança pode aumentar a complexidade e criar conflitos de métodos.

O conhecido **problema do diamante** ocorre quando uma classe herda de duas classes que, por sua vez, possuem uma mesma superclasse:

O MRO funciona como um “**caminho de busca**” que diz ao Python em qual ordem ele deve procurar um método até encontrar a implementação que será executada.

- Qual classe será consultada primeiro.
- Qual será consultada depois?
- Qual implementação do método será utilizada.
- Como evitar conflitos quando várias classes possuem métodos com o mesmo nome.

O Python utiliza uma regra chamada **C3 Linearization** para organizar essa ordem de forma consistente.

18. Propriedades em Python

`property` permite acessar um atributo de forma simples, mas mantendo **controle e regras sobre esse acesso**, contribuindo para o encapsulamento e a segurança do estado do objeto. **Ela está relacionada ao encapsulamento porque permite que possa ser consultado ou alterado, protegendo o estado interno do objeto.**

Porque o acesso direto pode permitir alterações inadequadas. Com uma propriedade, podemos:

- Validar os valores antes de aceitar uma alteração.
- Impedir valores inválidos.
- Permitir apenas a leitura de determinado atributo.
- Executar alguma regra antes de modificar o estado.

Exemplo:

Imagine um objeto que possui uma **idade**. Uma propriedade pode verificar se o valor informado é válido antes de alterar a idade, impedindo valores negativos.

19. Composição utilizando Python

No comércio eletrônico, `Pedido` pode possuir diversos `ItemPedido`.

```
class ItemPedido:
    def __init__(self, produto, quantidade):
        self.produto = produto
        self.quantidade = quantidade
```

```
class Pedido:
```

```

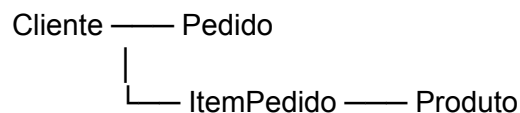
def __init__(self, cliente):
    self.cliente = cliente
    self.itens = []

def adicionar_item(self, produto, quantidade):
    self.itens.append(ItemPedido(produto, quantidade))

def calcular_total(self):
    return sum(
        item.produto.preco * item.quantidade
        for item in self.itens
    )

```

A relação é:



Pedido não é um tipo de **Cliente**, e **ItemPedido** não é um tipo de **Pedido**. Portanto, herança não representa adequadamente essas relações.

Composição representa melhor a ideia de que um pedido **é formado por** itens.

20. Padrão de projeto e Python — Strategy

O padrão **Strategy** permite encapsular diferentes algoritmos ou estratégias atrás de uma interface comum.

As classes participantes são:

- **Strategy**: define o contrato.
- **Concrete Strategies**: implementam os diferentes algoritmos.
- **Context**: utiliza a estratégia sem conhecer seus detalhes.

Entretanto, um padrão só deve ser introduzido quando resolve uma necessidade real. Aplicá-lo desnecessariamente pode aumentar a complexidade.

21. Princípio da Responsabilidade Única — SRP

O **Single Responsibility Principle (SRP)** estabelece que uma classe deve ter uma responsabilidade bem definida e, na formulação associada a Robert C. Martin, um único motivo para mudar.

Uma classe que cadastra clientes, valida dados, calcula descontos, gera relatórios e envia e-mails concentra responsabilidades distintas.

Uma possível reorganização:

ClienteService
└─ cadastro de clientes

ValidadorCliente
└─ validação

CalculadoraDesconto
└─ descontos

GeradorRelatorio
└─ relatórios

ServicoEmail
└─ envio de mensagens

Um serviço de aplicação pode coordenar essas classes:

```
ClienteService
  ↓
ValidadorCliente
  ↓
CalculadoraDesconto
  ↓
RepositorioCliente
  ↓
ServicoEmail
```

Isso permite modificar o mecanismo de e-mail, por exemplo, sem alterar a lógica de validação.

SRP não significa necessariamente que toda classe deva possuir apenas um método. A ideia central é manter responsabilidades relacionadas e motivos de mudança coerentes.

22. Princípio Aberto/Fechado — OCP

O **Open/Closed Principle (OCP)** significa que uma parte do sistema deve estar **aberta para receber novas funcionalidades**, mas deve evitar que seu código existente precise ser constantemente alterado.

Imagine um sistema de pagamento que possui uma única função com várias condições:

- Se for cartão, faz uma operação.
- Se for PIX, faz outra.
- Se for boleto, faz outra.

Sempre que uma nova forma de pagamento surgir, será necessário **alterar essa função**. Com o tempo, ela ficará grande, difícil de testar e mais sujeita a erros.

O **polimorfismo** ajuda porque permite que diferentes implementações sejam tratadas de maneira uniforme pelo código principal.

23. Acoplamento e coesão

- **Acoplamento** representa o grau de dependência entre componentes.
- **Coesão** representa o quanto as responsabilidades de um componente estão relacionadas entre si.

Uma arquitetura normalmente procura:

- **baixo acoplamento**: classes dependem pouco umas das outras;
- **alta coesão**: cada classe possui responsabilidades fortemente relacionadas.

Imagine uma classe **Pedido** que:

- salva os dados diretamente em um banco MySQL;
- envia e-mail diretamente pelo Gmail;
- calcula o pagamento diretamente;
- gera o relatório diretamente.

Nesse caso, **Pedido** depende de muitos componentes. Se o banco de dados ou o serviço de e-mail mudar, a classe **Pedido** também precisará ser modificada.

Pedido

└─ Repositório → responsável pelo banco

└─ Serviço de pagamento → responsável pelo pagamento

└─ Serviço de e-mail → responsável pelo envio

OBS:Podemos separar essas responsabilidades e fazer **Pedido** depender de **abstrações**, em vez de implementações específicas.

24. Interface versus implementação

Interface: apresenta os métodos e comportamentos que podem ser utilizados.

Implementação: contém o código e os detalhes necessários para realizar esses comportamentos.

Por que separá-las ?

Porque o restante do sistema pode utilizar uma classe **sem precisar conhecer como ela funciona internamente**.

Isso facilita a evolução porque podemos **alterar ou substituir a implementação** sem precisar modificar todo o código que utiliza aquela interface.

25. Inversão de Dependência — DIP

O **Dependency Inversion Principle (DIP)** significa que uma classe importante do sistema **não deve depender diretamente de uma implementação específica**. Em vez disso, ela deve depender de uma **abstração**.

Problema

Imagine que a classe **Pedido** dependa diretamente de um banco **MySQL**.

Isso gera problemas porque, se for necessário trocar o MySQL por PostgreSQL, MongoDB ou outro mecanismo, será necessário alterar a classe **Pedido**.

Isso aumenta o **acoplamento** e dificulta a manutenção e os testes.

Solução

Podemos criar uma abstração, como um **repositório**, que define as operações de persistência.

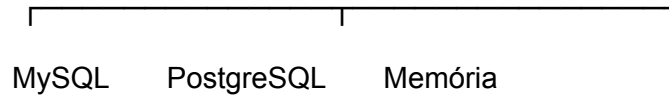
A estrutura seria:

Pedido

↓

Repositório (abstração)

↓



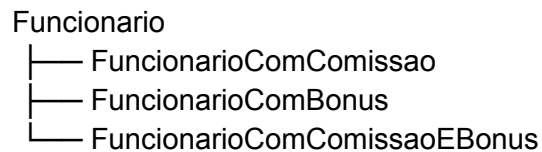
A classe `Pedido` conhece apenas o **Repositório**, e não um banco específico.

Assim, podemos trocar o mecanismo de persistência sem modificar significativamente `Pedido`.

26. Herança versus composição

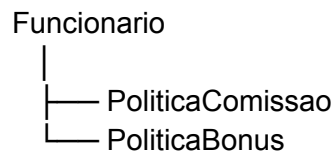
A recomendação "**prefira composição à herança quando a relação de especialização não for realmente necessária**" está relacionada ao fato de que herança cria uma dependência estrutural forte entre classe-pai e subclasses.

Imagine inicialmente:



À medida que novas combinações aparecem, a hierarquia pode crescer rapidamente.

Com composição:



O funcionário pode receber diferentes comportamentos:

```
class Funcionario:
    def __init__(self, comissao, bonus):
        self.comissao = comissao
        self.bonus = bonus
```

Assim, comportamentos podem ser combinados independentemente.

Herança é apropriada quando existe uma verdadeira relação de especialização e o contrato da superclasse faz sentido para todas as subclasses. Composição é interessante quando o objetivo principal é combinar comportamentos ou representar relações de "possui".

27. Modelagem orientada a objetos — Sistema universitário

Classe	Atributos	Métodos
Aluno	nome, matrícula, e-mail, curso	matricular(), cancelar_matricula(), consultar_notas()
Professor	nome, registro, departamento	atribuir_nota(), ministrar_aula()
Curso	nome, código, carga_horária	adicionar_disciplina()
Disciplina	código, nome, carga_horária	—
Turma	código, período, disciplina, professor	—
Matrícula	aluno, turma, data, status	—
Avaliação	descrição, nota, peso	—

Possíveis relacionamentos:

Curso — Disciplina
Professor — Turma
Disciplina — Turma
Aluno — Matrícula
Turma — Matrícula
Turma — Avaliação

Associações: aluno participa de uma turma por meio de matrícula; professor ministra uma turma.

Composição: uma **Turma** pode ser modelada como composta por suas avaliações quando estas não possuem significado independente fora da turma.

Agregação: um curso pode agregar disciplinas que também podem existir em outras estruturas acadêmicas, dependendo das regras do domínio.

Herança: poderia existir uma abstração **Pessoa**, com **Aluno** e **Professor** como especializações, caso compartilhem identidade e comportamentos relevantes.

OBS: É importante observar que UML e POO não impõem uma única modelagem correta. A escolha depende das regras do domínio.

28. POO e manutenção de sistemas legados

Migrar um sistema procedural grande para POO apresenta diversos riscos:

- quebra de funcionalidades existentes;
- dependência de variáveis globais;
- funções fortemente acopladas;
- dificuldade para identificar responsabilidades;
- falta de testes automatizados;
- regras de negócio espalhadas;
- dificuldade para separar dados e comportamentos.

Uma estratégia segura é realizar uma **migração incremental**, evitando reescrever todo o sistema de uma vez.

Uma possível sequência:

1. **Mapear o sistema existente.**
2. Identificar funcionalidades e dependências críticas.
3. Criar testes de caracterização para registrar o comportamento atual.
4. Identificar conceitos importantes do domínio.
5. Criar classes que encapsulem partes específicas.
6. Introduzir interfaces entre código novo e legado.
7. Substituir gradualmente funções e variáveis globais.
8. Executar testes continuamente.
9. Migrar módulos isoladamente.
10. Remover gradualmente código antigo que deixou de ser utilizado.

Um padrão como **Strangler Fig** pode ser utilizado conceitualmente: funcionalidades novas ou modificadas são implementadas na nova arquitetura enquanto partes antigas continuam funcionando até serem substituídas.

O objetivo não é simplesmente transformar funções em métodos, mas **reduzir dependências e reorganizar responsabilidades de forma gradual**.

29. POO, arquitetura e escalabilidade

Em sistemas grandes, POO pode contribuir para controlar complexidade por meio de vários mecanismos.

Abstração: Permite trabalhar com conceitos essenciais sem expor detalhes internos.

Encapsulamento : Protege invariantes e reduz dependências sobre detalhes de implementação.

Polimorfismo: Permite substituir ou adicionar implementações sem alterar necessariamente o código consumidor.

Baixo acoplamento: Reduz o impacto das mudanças entre componentes.

Alta coesão: Mantém responsabilidades relacionadas agrupadas.

Reutilização: Composição, abstrações e componentes bem projetados podem evitar duplicação.

SOLID: Os princípios SOLID oferecem diretrizes para organização e evolução:

- **SRP:** responsabilidades bem delimitadas.
- **OCP:** extensão sem modificações constantes.
- **LSP:** subclasses devem respeitar o contrato esperado.
- **ISP:** interfaces devem evitar obrigar clientes a depender de operações desnecessárias.
- **DIP:** dependências importantes devem apontar para abstrações.

Esses princípios podem ajudar em sistemas corporativos nos quais diferentes equipes precisam modificar partes do sistema sem causar efeitos inesperados em outras partes.

Entretanto, escalabilidade arquitetural não depende somente de POO. Também envolve arquitetura de módulos, banco de dados, comunicação entre serviços, observabilidade, testes, infraestrutura, desempenho e organização do desenvolvimento.

30. Análise crítica de uma arquitetura orientada a objetos

- Utilizar muitos conceitos avançados de POO não significa automaticamente produzir uma boa arquitetura. Um sistema pode possuir dezenas de interfaces, herança múltipla e vários padrões de projeto e, mesmo assim, apresentar grande complexidade.
- Um problema possível é a **complexidade acidental**: dificuldades introduzidas pela própria solução, e não pelo problema de negócio.

Abstrações desnecessárias

Abstrações são úteis quando isolam variações reais. Criá-las apenas para seguir uma regra de forma mecânica pode dificultar a compreensão.

Overengineering: Overengineering ocorre quando a solução possui complexidade maior que a necessária para resolver o problema.

Acoplamento: Muitas classes não significam necessariamente baixo acoplamento. Classes podem continuar dependendo fortemente umas das outras, mesmo quando existe uma grande quantidade de interfaces.

Coesão : Uma classe deve possuir responsabilidades relacionadas. Dividir excessivamente um comportamento pode gerar classes pequenas, mas semanticamente fragmentadas.

Legibilidade : Código orientado a objetos deve continuar compreensível. O desenvolvedor precisa conseguir identificar facilmente:

- onde está a regra de negócio;
- quem é responsável por cada operação;
- quais objetos colaboram;
- quais dependências existem;
- quais componentes podem ser substituídos.

SOLID

SOLID deve ser utilizado como conjunto de princípios de design, não como uma obrigação de criar interfaces e classes para tudo.

Critérios para avaliar a arquitetura

Uma arquitetura pode ser analisada considerando:

1. **Clareza:** o código é fácil de compreender?
2. **Coesão:** cada componente possui responsabilidades relacionadas?
3. **Acoplamento:** mudanças em um componente exigem alterações em muitos outros?
4. **Testabilidade:** componentes podem ser testados isoladamente?
5. **Extensibilidade:** novas necessidades podem ser incorporadas sem alterações excessivas?
6. **Complexidade:** cada abstração possui uma justificativa concreta?
7. **Legibilidade:** a estrutura facilita ou dificulta a leitura?
8. **Manutenibilidade:** mudanças comuns podem ser realizadas com segurança?
9. **Adequação ao domínio:** o modelo representa conceitos reais do problema?
10. **Simplicidade:** a arquitetura utiliza a menor complexidade capaz de atender aos requisitos?

Portanto, uma boa arquitetura orientada a objetos não é aquela que utiliza **mais** conceitos de POO, mas aquela que utiliza abstrações e mecanismos de orientação a objetos de maneira **proporcional às necessidades do sistema**. O objetivo principal é controlar a complexidade e facilitar a evolução do software.